Redis Configuration

The application defines a Spring `RedisConfiguration` that builds a `RedisCacheManager` with a default entry TTL of 1 day (via `.entryTtl(Duration.ofDays(1))`). In theory, this means any cache entries managed by this `RedisCacheManager` would expire after one day. However, note that in the code the `redisCacheManager` method is not annotated with `@Bean`, and **the application does not use Spring's `@Cacheable` mechanism**. Instead, it uses a low-level `RedisTemplate`. As a result, the default TTL setting in `RedisConfiguration` is **not actually applied** to the entries created by `CacheServiceImpl`.

Cache Service Layer

The `CacheServiceImpl` class handles storing and retrieving cached templates in Redis. It uses a Spring `RedisTemplate<String,String>` and stores JSON-serialized templates in a **Redis hash** per tenant. The key for each tenant's cache is constructed as `TEMPLATE_REDIS_PREFIX + tenantId` (i.e. `"template." + tenantId`). Within that hash, it uses two different field names for each template: one for lookups by ID and one for lookups by name.

- **Key structure:** For a tenant with ID `"tenant_1"`, the Redis key would be `"template.tenant_1"`. Inside this hash, each template is stored under two fields: `"BY_ID:<id>"` and `"BY_NAME:<name>"`. For example, a template with ID `101` and name `"Welcome"` would be stored as fields `BY_ID:101` and `BY_NAME:Welcome` in the `"template.tenant_1"` hash.

- **Storing (put):** The generic `put(tenantId, hashKey, data)` method serializes the `data` object to a JSON string (`objectMapper.writeValueAsString(data)`) and then calls `redisTemplate.opsForHash().put(mainKey, hashKey, json)`. The convenience methods `putBYid` and `putByName` simply call `put(...)` with the appropriate `"BY_ID:"` or `"BY_NAME:"` prefix concatenated to the given ID or name. For example, `putBYid("tenant_1", "101", template)` will store the JSON under field `"BY_ID:101"` in the `"template.tenant_1"` hash.

- **Retrieving (get):** Similarly, `getByID` and `getByName` call a generic `get(tenantId, hashKey, clazz)` which does `opsForHash().get(mainKey, hashKey)`. It then parses the returned JSON string back into an object (`objectMapper.readValue(jsondata, clazz)`). If a JSON value is present, it

returns it wrapped in `Optional.ofNullable`; if the field is absent, `opsForHash().get` returns null and the method ultimately returns `Optional.empty`.

- **Deleting:** The `deleteById` and `deleteByName` methods call `redisTemplate.opsForHash().delete(mainKey, hashKey)` to remove the specific field. (They do not remove the entire tenant key, only the individual field(s).)

In short, the cache service uses one Redis hash per tenant, with separate fields `"BY_ID:<id>"` and `"BY_NAME:<name>"` mapping to the JSON data. This effectively provides *multi-key lookup* within a single Redis key (see “Multi-Key Lookups” below).

DAO Layer (TemplateDaoImpl)

The `TemplateDaoImpl` class orchestrates reads and writes, combining MongoDB access with the cache:

- **`findByTenantIdAndName(tenantId, name)`:** It first calls `cacheService.getByName(tenantId, name, Template.class)`. If the cache returns a present `Optional<Template>`, it immediately returns that (cache hit). If the cache is empty, it queries Mongo via `templateRepository.findByNameIgnoreCaseAndTenantId(...)`. If the database returns a result, it then calls `cacheService.putByName(tenantId, name, template)` to cache it, and finally returns the DB result.

- **`findByTenantIdAndId(tenantId, id)`:** This method uses `Optional.or(...)` logic. It does `cacheService.getByID(tenantId, id, Template.class).or(() -> templateRepository.findByTenantIdAndId(...).map(template -> {...}))`. In practice, that means: try the cache first (and if a Template is found, return it). On a cache miss, query Mongo with `findByTenantIdAndId`. If Mongo finds the template, the code calls `cacheService.putBYid(tenantId, id, template)` to cache it before returning it. (Note: unlike `findByName`, this block only caches the “by ID” field, not the name field.)

- **`save(template)`:** This saves the template to Mongo (`templateRepository.save(...)`) and then **updates both cache entries**. It calls `cacheService.putByName(tenantId, template.getName(), saved)` and `cacheService.putBYid(tenantId, id, saved)`. This ensures that *both* the

name-based and ID-based lookup keys in the hash are set (in case the template was new or updated).

- **`deleteTemplateById(id)`:** This method looks up the template (via `findByTenantIdAndId`, which itself uses the cache/DB logic). If not found, it returns a "not found" message. If found, it deletes the entity from Mongo (`templateRepository.deleteById(...)`). It then clears the cache entries for that template: it calls `cacheService.deleteByName(tenantId, template.getName())` and `cacheService.deleteById(tenantId, id)` to remove both fields from the tenant's hash. Thus, after deletion, neither lookup will return the stale template.

Overall, **the DAO layer first checks Redis cache, falls back to Mongo, and then populates the cache on a miss**. The cache is updated on saves, and cleared on deletes, to keep the cache in sync with the database.

Multi-Key (Name/ID) Lookups

The implementation uses *two different hash fields* per template to allow lookup by either name or ID. Specifically:

- When a template is cached, it is stored *twice* within the same Redis hash: once under `"BY_ID:<id>"` and once under `"BY_NAME:<name>"`. For example, after saving a template with ID `123` and name `"Welcome"`, the tenant's Redis hash will have a field `BY_ID:123 → "<json>"` and a field `BY_NAME:Welcome → "<json>"`. (Both fields contain the same JSON data.)

- The `CacheServiceImpl` methods `putBYid` and `putByName` implement this by simply concatenating the respective prefix and key: `putBYid(tenantId, id, data)` calls `put(tenantId, "BY_ID:" + id, data)`, and similarly for name. The retrieval methods `getByID` and `getByName` use the same prefixes.

- In the DAO, after saving or loading from the database, the code takes care to set both fields (in `save`) or at least the relevant field (in find-by-name, it sets name; in find-by-id, it sets id). On delete, it deletes *both* fields so that neither lookup returns the deleted template.

This strategy ensures that a template can be quickly retrieved from cache by either its ID or its name without needing separate Redis keys for each. All lookups go to the same tenant-specific Redis hash.

TTL (Time To Live) Handling

The code as written **does not apply a Redis TTL to the cached data**, despite the attempt in `RedisConfiguration` to set a 1-day default. The key points are:

- The `RedisConfiguration` class builds a `RedisCacheManager` with `.entryTtl(Duration.ofDays(1))`, but that bean is not actually used in `CacheServiceImpl`. The cache service directly uses `RedisTemplate` and writes to a hash. Spring's `RedisCacheManager` (and any `@Cacheable` annotation) is never used in this code path.

- **No explicit expiration:** The code never calls `redisTemplate.expire(...)` on the tenant hash key. All `opsForHash().put` or `delete` calls simply update the hash. By default, Redis keys in this setup will remain indefinitely (until the application explicitly deletes them).

- **Implications:** Because TTL is not applied, cache entries will **persist forever** or until manually removed. This means:

    - If a template is updated in Mongo outside the code's context, the cache will hold the old version indefinitely (stale data).

    - Memory usage may grow if many templates/tenants are cached and never expired.

    - The intended "1 day" TTL from configuration is effectively not in effect.

- **Correct TTL usage:** If we wanted TTL, we would need to explicitly set it on the Redis key (e.g. `redisTemplate.expire(getTenantCacheKey(tenantId), 1, TimeUnit.DAYS)` after each put). Note that since the code uses a single hash per tenant, this would set a 1-day expiry on the whole hash (all fields). Alternatively, storing each template under its own Redis key (instead of a hash) would allow individual TTLs on each template. As is, the default TTL is not applied, so the cache service treats the tenant hash as **permanent**.

End-to-End Flow Example

Putting it all together, here is how a typical “read-or-create” flow works for a template:

1. **Client requests a template (by name or ID)** and the application calls the DAO, e.g. `findByTenantIdAndName(tenantId, name)`.

2. **Cache lookup:** Inside `findByTenantIdAndName`, it calls `cacheService.getByName(tenantId, name, Template.class)`. This does `redisTemplate.opsForHash().get("template."+tenantId, "BY_NAME:"+name)`.

    - If Redis returns JSON, it is parsed and returned immediately.

    - If Redis returns null (cache miss), the DAO queries Mongo (`templateRepository.findByNameIgnoreCaseAndTenantId(...)`).

3. **Database fallback:** If Mongo finds the template, DAO then caches it by calling `cacheService.putByName(tenantId, name, template)`. This serializes the template to JSON and stores it in the Redis hash under field `"BY_NAME:"+name`. (Note: in `findByName` the code does *not* immediately call `putBYid`, so the `"BY_ID:"` field is not set in this path.)

4. **Return result:** The DAO returns the template (from cache or DB) to the caller.

On subsequent requests:

- If another component calls `findByTenantIdAndId(tenantId, id)`, the DAO will try `cacheService.getByID(tenantId, id, Template.class)` first. If the `"BY_ID:"+id` field was not set yet, it will be a cache miss and it will fetch from DB and then call `putBYid(tenantId, id, template)`, storing under that field.

- Eventually, after a save or multiple lookups, both fields for that template will be in the cache. For example, after calling `save(template)`, the code explicitly sets *both* `"BY_NAME"` and `"BY_ID"` fields, ensuring consistent lookups.

When a template is **deleted**, `deleteTemplateById(id)` does a DB delete and then removes both cache fields: `deleteByName` and `deleteById`. After this, the tenant’s Redis hash still exists, but with those fields cleared.

TTL & Expiration Implications

As noted, TTL is *not* actually applied to the cached entries in this implementation. The hash key `"template.{tenantId}"` will **not expire** unless manually deleted. The only deletion that occurs is when individual templates are deleted (their fields are removed). Thus:

- **No automatic eviction:** Cached templates remain until a delete call or the application code explicitly removes them.
- **Stale data risk:** Without a time-based expiration, if template data changes in Mongo (outside this code path), the cache could serve outdated data indefinitely.
- **Memory usage:** All tenant hashes accumulate entries. If many tenants/templates exist, memory could grow until manual clearing or server restart.